



Comp90042

Natural Language Processing

Workshop (Week2 – 2026s1)

Dr Jean Lee





About Jean <jeanlee@unimelb.edu.au>



Associate Lecturer

PhD (Computer Science - NLP), Master of Data Science

Research Area

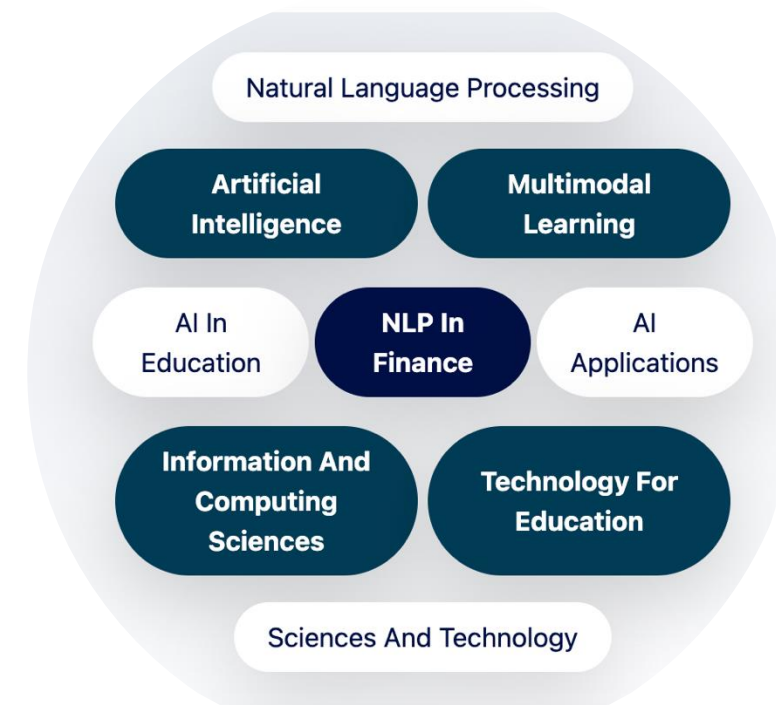
- **AI, Computational Linguistics, Information Retrieval**

Teaching (School of Computing & IS and Business School)

- Computing, Machine Learning, Natural Language Processing
- Data Science, Data Analytics
- ***FEIT Education Excellence Awards – Tutor & GRAA (2025)***
- ***FEIT Community Awards – Tutor of the Year (2025)***
- **NVIDIA LLM Bootcamp** with CSIRO's National AI Centre
- **GoogleExploreCSR @ University of Sydney**

Prior to joining Academia

- AICPA, MBA, Data Scientist
- Management Consultant @Accenture, KPMG





About Assessment

COMP90042 2026 SM1 > Modules > Lectures > Slides > Week 1 Lecture 1

- **Assignments** (17% total for 3 activities)
 - 2 programming exercises
 - Released in week 3 and 5; 1 week to complete
- **Group Project** (35%)
 - Released in week 6/7; 4 weeks to complete
- **Peer Review** (8%)
 - Released in week 12
- **Exam** (40%)
 - 2 hours, on-campus, closed book
 - Covers content from lectures, workshop
- **Hurdle** >50% exam (20/40), **AND** >50% for assignments + project + peer review (30/60)



About Workshop

COMP90042 2025 SM1 > Modules > Workshops > Worksheets

- **Discussion**: regarding key concepts in the lectures (~ 30mins)
 - Worksheet (.pdf file)
- **Programming**: linking theory to practice (~ 30mins)
 - Jupyter notebook on Google Colab (.ipynb file)
- *We are not able to cover all questions in one-hour workshop. Please make the most of **Ed Discussion**.*
- *The Solution to the tutorial will be released on **Next Week**.*

Date	Week	Worksheets	Notebooks	Solutions
9 Mar	2	 workshop-02.pdf 	01-preprocessing.ipynb  02-bpe-1.ipynb 	
16 Mar	3	 workshop-03.pdf 	03-classification.ipynb  04-ngram.ipynb 	

Jupyter Notebook on Google Colab

COMP90042 2026 SM1 > Modules > Resources > Using Jupyter Notebook...

Using Jupyter Notebook and Python
on Google Colab 

<https://colab.research.google.com/>

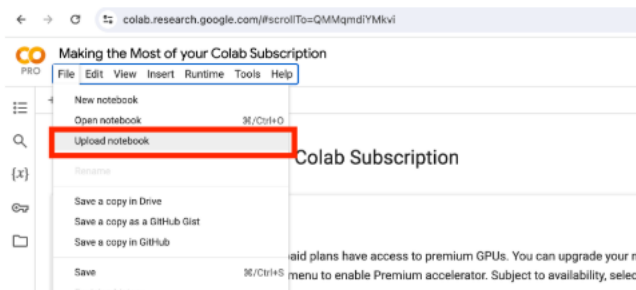
Colab Introduction

We will use Google Colab which provides access to a Jupyter notebook environment with all the required packages pre-installed. Feel free to play around with the notebook to familiarise yourself with Colab.

1. Go to Colab webpage

Please go to <https://colab.research.google.com>  and ensure that you can log in with your Unimelb account. You should be greeted with a "Welcome To Colab" notebook when you can log in successfully.

2. Upload your .ipynb file





Agenda

1. Icebreaker with your peers
2. Discussion on worksheet questions
3. Notebook on programming

At the end of this workshop you should:

- *be familiar with the **Tokenisation** and **Normalisation** (stemming and lemmatisation)*
- *be familiar with **Byte-pair Encoding(BPE)** algorithm, and be able to implement it*



Icebreaker

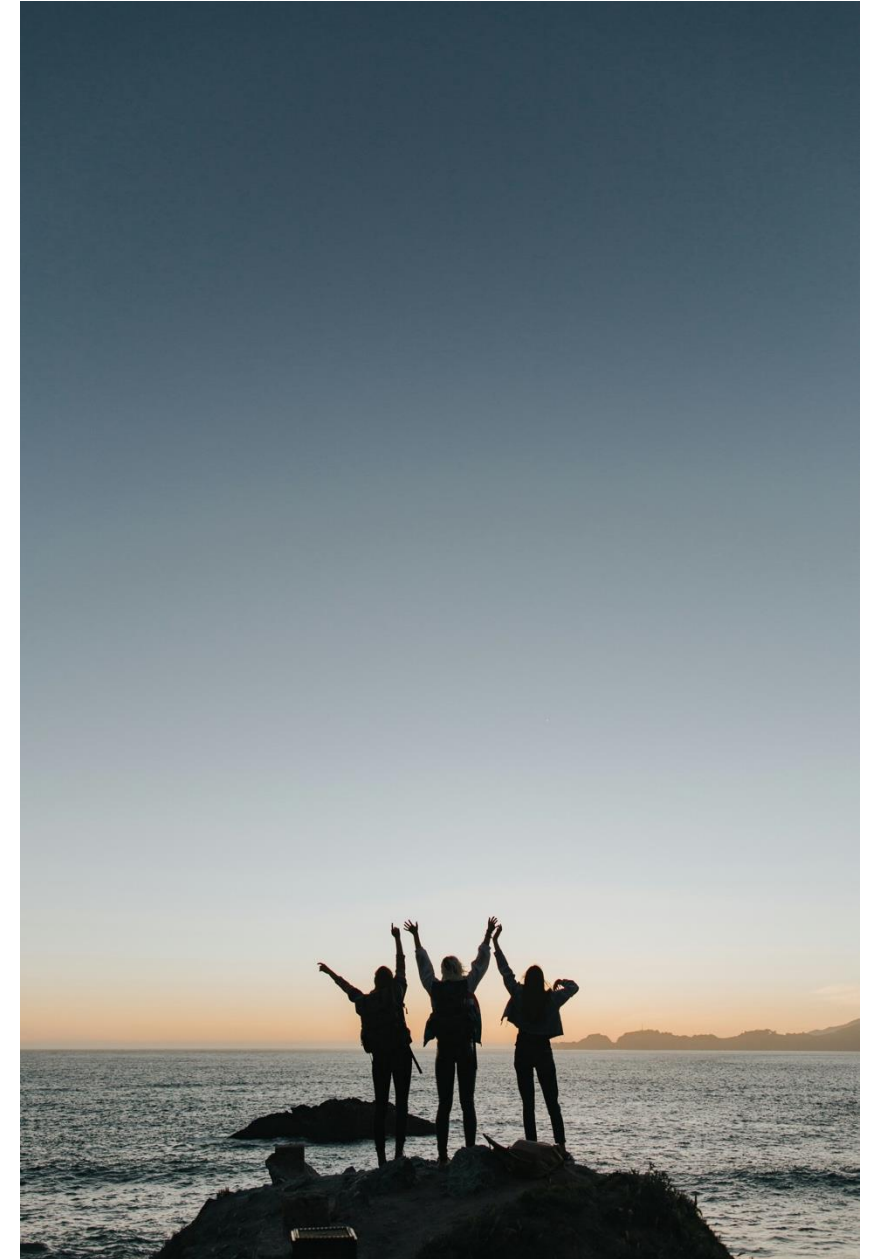
Get into groups of 2-4 and Introduce yourself

- What is your name?
- What are you studying?
- Why choose COMP90042?

Then, Discuss the first question of the worksheet

- Give some examples of text processing applications that you use on a daily basis.

*You should start to know each other as future teammates for the **project** and **peer review** task.*





Discussion

- Examples of text processing applications

Possible answers

- Web search engines (Google, Copilot, Bing, Perplexity,...)
- Speech-to-text systems (Siri, Alexa, Google, ...)
- Spelling correction (Grammarly, ...)
- Machine translation (Google, DeepL, ...)

Discussion - Examples of text processing applications (GenAI)

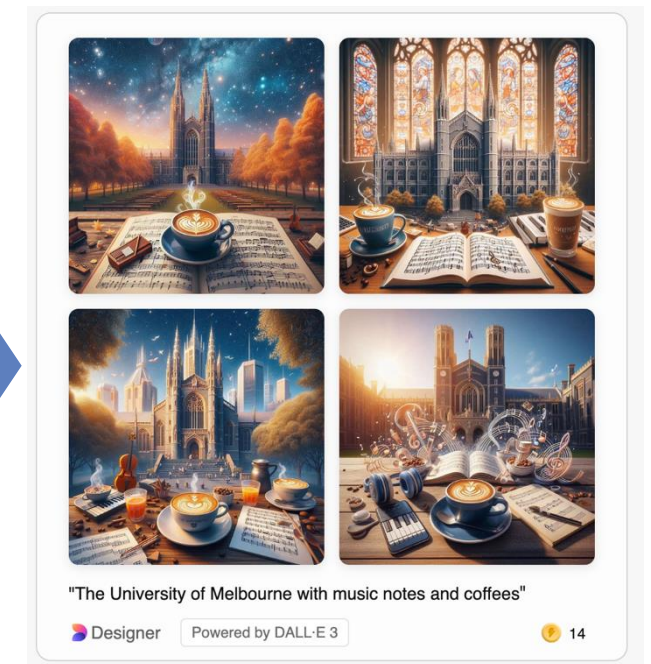
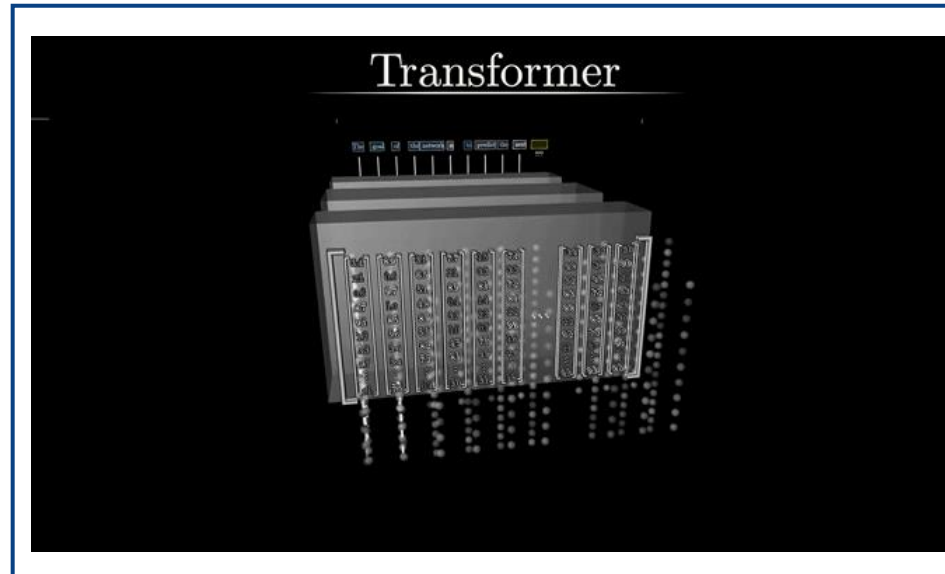
text-to-image
text-to-code
text-to-speech
speech-to-text

⋮



generate an image

"The University of Melbourne with music notes and coffee"



Discussion

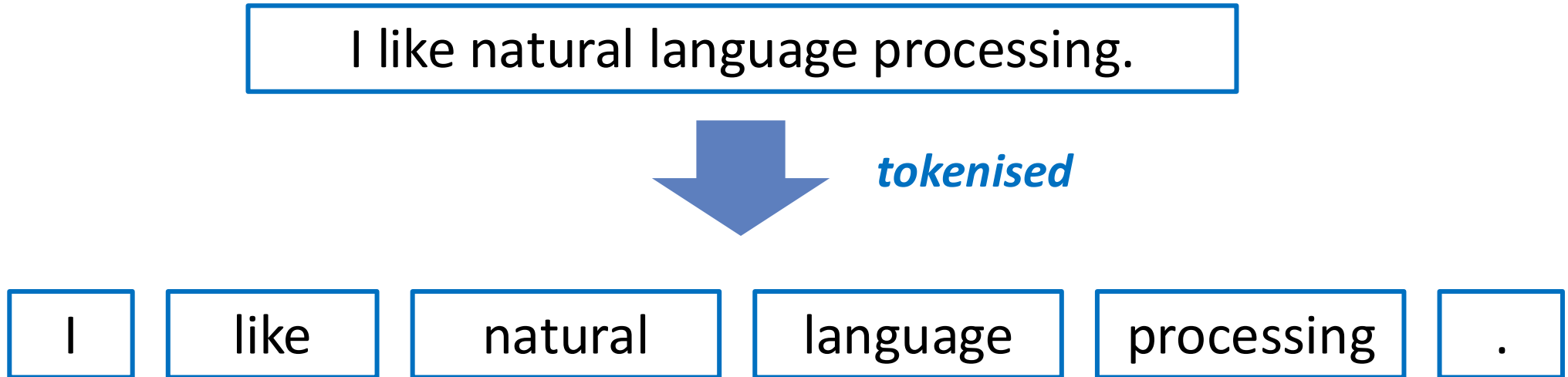
Discussion

1. Give some examples of text processing applications that you use on a daily basis.
2. What is **tokenisation** and why is it important?
 - (a) What are **stemming** and **lemmatisation**, and how are they different? Give examples from the 01-preprocessing iPython notebook.

Discussion - Word Tokenisation

What is tokenisation?

- Segmenting text into tokens (words/subwords)



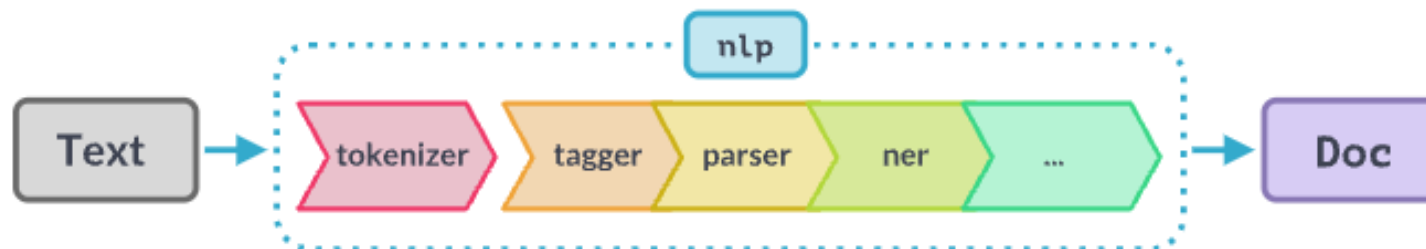
Discussion - Word Tokenisation

What is tokenisation?

- Segmenting text into tokens (words/subwords)

Why is it important?

- A document may too long to manipulate directly.
- Easier for machine to understand.
- Human can break it into individual components, machine should do the same.



Discussion - Word Tokenisation

What are stemming and lemmatisation?

(Quiz)

- Computers -> Computer : **Lemmatisation** *removing any inflection to reach the uninflected form, the **lemma***
- Computers -> Comput : **Stemming** *strips off all suffixes, leaving a **stem***

What are Pros & Cons?

- Pros (+): Keep the **semantic information** at some level (*meaning of words and phrases*)
Keep **efficiency** for the downstream task
- Cons (-): Loss of **contextual information** (*surrounding environment that influence meaning*)

Discussion - Word Tokenisation

Inflectional and Derivational Morphology

“form, shape”

-> the study of the internal structure of words

(Quiz)

- Teacher -> Teachers : **inflectional morphology**

*is the systematic process by which tokens are **altered** to conform to certain **grammatical constraints**. **don't alter the meaning of the term**.*

(e.g.) teacher (singular) -> teachers (plural)

- Teach -> Teacher : **derivational morphology**

*is the (semi-)systematic process by which we **transform** terms of one class **into a different class**. Both listed in the **lexicon**.*

(e.g.) teach (verb) -> teacher (noun)

Discussion - Word Tokenisation

How are they different?

- **Lemmatisation** : removing any inflection to reach the uninflected form, the *lemma*
- **Stemming** :strips off all suffixes, leaving a *stem*

	Stemming	Lemmatisation
May output garbage tokens	✓	
Remove inflectional morphology	✓	✓
Usually remove derivational morphology	✓	
Works with a lexicon (a list of valid words)		✓
Remove or replace affixes (primarily suffixes)	✓	✓
Transform a token into a normalised form	✓	✓

Programming!

1. Make sure that you have a Python environment where you can run the given iPython notebooks. In particular, ensure that the `numpy`, `sklearn` and `nltk` packages are installed (i.e. you can `import` them).
2. Adapt the `01-preprocessing` iPython notebook into a program which tokenises an input file based on the five-step model given in the lectures.
3. Complete the BPE tokenisation algorithm in the `02-bpe` iPython notebook.

Stemming and Lemmatisation

Examples from the 01-preprocessing notebook

'Topics to be covered include part-of-speech tagging, n-gram language modelling, syntactic parsing and deep learning.'

```
1 word_tokenizer = nltk.tokenize.regexp.WordPunctTokenizer()
2 tokenized_sentence = word_tokenizer.tokenize(sentences[1])
3 print(tokenized_sentence)
```

```
['Topics', 'to', 'be', 'covered', 'include', 'part', '-', 'of', '-', 'speech', 'tagging', ',', 'n', '-', 'gram',
```

```
1 print(sentences[1].split(" "))
```

```
['Topics', 'to', 'be', 'covered', 'include', 'part-of-speech', 'tagging,', 'n-gram',
```

```
10 print([lemmatize(token) for token in tokenized_sentence])
```

```
['Topics', 'to', 'be', 'cover', 'include', 'part', '-', 'of', '-', 'speech', 'tag', ',', 'n', '-', 'gram',
```

```
1 stemmer = nltk.stem.porter.PorterStemmer()
2 print([stemmer.stem(token) for token in tokenized_sentence])
```

```
['topic', 'to', 'be', 'cover', 'includ', 'part', '-', 'of', '-', 'speech', 'tag', ',', 'n', '-', 'gram',
```

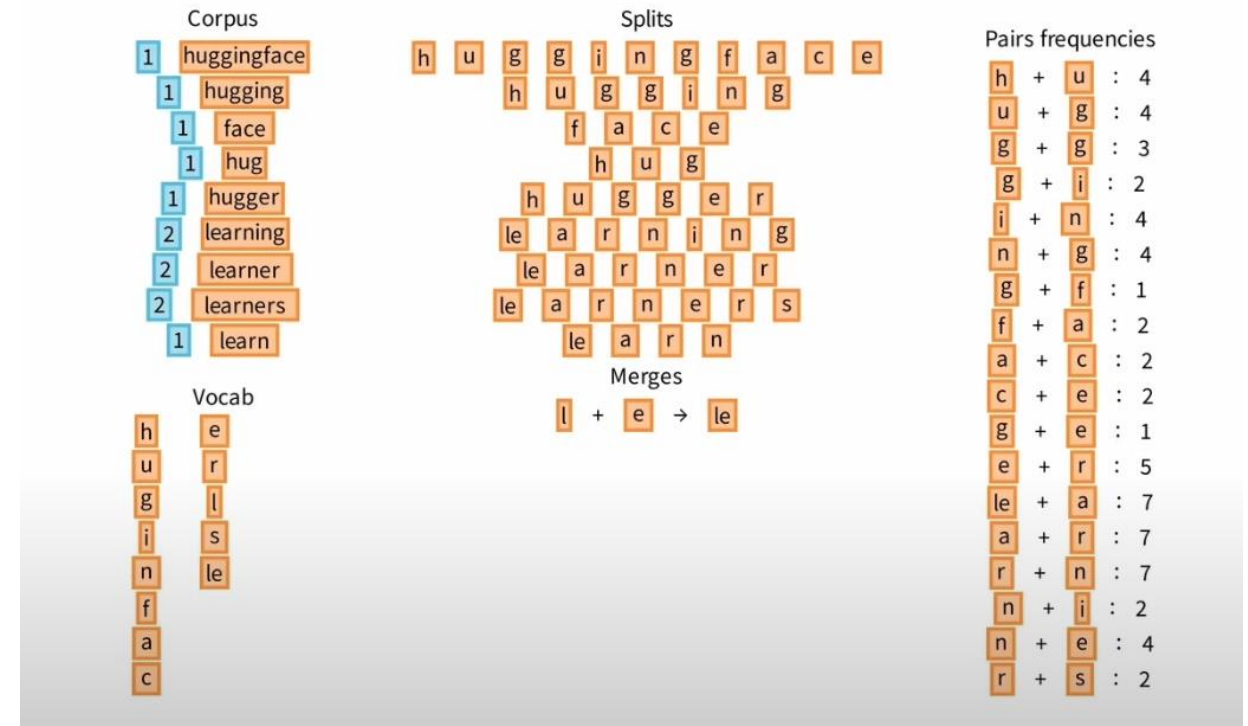
Byte-Pair Encoding (BPE)

What is Byte-Pair Encoding (BPE)?

- a subword tokenization algorithm
- iteratively merge frequent pairs of characters

Advantage of BPE?

- Data-informed tokenisation
- Works for different languages
- Deals better with unknown words



Byte-Pair Encoding (BPE)

Examples from the 02-bpe notebook

```
text = "The aims for this subject is for students to develop an understanding of the  
main algorithms used in natural language processing,..."
```

```
Vocab = defaultdict(<class 'int'>, {'T h e </w>': 2, 'a i m s </w>': 1, 'f o r </w>  
=====
```

Tokens Before BPE

```
Tokens: defaultdict(<class 'int'>, {'T': 3, 'h': 11, 'e': 39, '</w>': 73, 'a': 38,  
Number of tokens: 31  
=====
```



*Byte-Pair Encoding
tokenisation algorithm*

```
Iter: 99  
Best pair: ('nat', 'u')  
Tokens: defaultdict(<class 'int'>, {'T': 1, 'h': 4, 'e': 8, '</w>': 11,  
Number of tokens: 131  
=====
```



*Use BPE dictionaries
to tokenise sentences*

```
sentence_1 = 'I like natural language processing!'  
sentence_2 = 'I like natural languaaage processing!'
```

```
def get_vocab(text):  
def get_stats(vocab):  
def merge_vocab(pair, v_in):  
def get_tokens(vocab):  
num_merges = 100
```

```
Tokenizing word: language</w>...  
['language</w>']
```

```
Tokenizing word: languaaage</w>...  
['langu', 'a', 'a', 'ag', 'e</w>']
```